浅谈概率数据结构

概率数据结构 (Probabilistic Data Structure, PDS) 是一种依赖随机性质来获取相较确定性数据结构更优秀的时空效率的数据结构. 其核心要义是通过允许对询问的回答以一定的概率错误, 来极大地简化存储或计算成本. 当对答案的精确性要求不高, 或者精确答案的求解开销过大时, PDS 是一种适合的解决手段. 其在大数据和大模型等领域有较多的应用.

本文试介绍 Bloom Filter, Four-colored Filter 和 Fermat Sketch, 前者是经典的 PDS, 后两者或许相对少见; 最后略作思考补充. 由于相关资料 (或者笔者的论文检索能力) 有限, 对 Four-colored Filter 和 Fermat Sketch 的介绍只基于信概课程的粗略笔记, 再由笔者重新推导成形, 可能有偏颇错漏, 还望批评指正.

1 Bloom Filter

Bloom Filter 是一种经典的 PDS, 它可以用于处理以下问题:

> **问题 1.1**
>
> 给定集合 S, 回答若干次询问, 每次询问给出元素 x, 回答是否有 $x \in S$.

传统做法无非是使用 hash table 或者平衡树. 在数据量极大时, 它们都有较大的开销.

> **算法 1.2 (Bloom Filter)**
>
> 对 问题 1.1, 选定 hash 函数值域 $[0:m-1]$ 和 k 个不同且相互独立的 hash 函数 $\{h_k\}$ 来维护一个长为 m, 初始全 0 的 bit array, 以支持对 S 的元素插入和在线查询.
>
> - 插入元素 x 时: 将 bit array 中 $h_1(x), \cdots, h_k(x)$ 位置对应的 bit 全部置为 1.
> - 查询元素 y 时: 检查 bit array 中 $h_1(y), \cdots, h_k(y)$ 位置对应的 bit 是否全部为 1. 若全为 1, 回答 "$y \overset{?}{\in} S$", 否则回答 "$y \notin S$".

容易看出, Bloom Filter 只可能给出假阳性结果. 即, 若 Bloom Filter 认为 $y \notin S$, 则 $y \notin S$ 事实上一定成立; 若 Bloom Filter 认为 $y \overset{?}{\in} S$, 事实上不一定有 $y \in S$, 因为 $h_1(y), \cdots, h_k(y)$ 这些 bit 可能被其他元素的 hash 值命中, 而产生错误.

为了平衡效率和正确率, 我们需要对 m 和 k 进行适当调节. 作为例子, 这里仅考察 m 和 $|S| = n$ 固定时对 k 的选取, 同时假设 hash 函数的取值皆均匀随机. 在 n 个元素插入完成后, 单个 bit 处于 0 的概率为 $p_0 = \left(1 - \frac{1}{m}\right)^{kn}$. 在 m 充分大时, 根据

$$\lim_{m \to \infty} \left(1 - \frac{1}{m}\right)^m = \frac{1}{e},$$

可化简得

$$p_0 \approx e^{-kn/m}.$$

则 Bloom Filter 的假阳性概率可以估计为

$$\varepsilon \approx (1 - p_0)^k \approx \left(1 - e^{-kn/m}\right)^k.$$

令 $q = n/m$, 那么

$$\ln \varepsilon = k \ln \left(1 - e^{-kq}\right)$$
$$\Rightarrow \frac{\mathrm{d} \ln \varepsilon}{\mathrm{d}k} = \ln \left(1 - e^{-kq}\right) + \frac{kqe^{-kq}}{1 - e^{-kq}}.$$

令 $t = e^{-kq}$, 则 $kq = -\ln t$; 令导数取 0, 则在极值点处有

$$\ln(1-t) - \frac{t \ln t}{1-t} = 0 \Leftrightarrow (1-t)\ln(1-t) = t \ln t.$$

其中 $t \in (0,1)$, 由单调性不难看出当且仅当 $t = \frac{1}{2}$ 时取等. 这时

$$e^{-kn/m} = \frac{1}{2} \Rightarrow k = \frac{m}{n} \ln 2.$$

此外, 在给定期待的误差率 ε 时, 也能给出相应的 m 选取, 相关计算这里略去. 经此例可以看出, 诸如 Bloom Filter 这样的 PDS 内部具有较大的自由度, 因此有必要针对参数优化问题 (甚至结合硬件效率, 成本等因素) 进行充分的数学处理.

2 Four-colored Filter

在某种意义上, Four-colored Filter 是在一个限制更强的集合判属问题上对 Bloom Filter 的扩展.

> **问题 2.1**
>
> 给定 n 个无交集 $S_1, \cdots, S_n$, 设 $S = \bigsqcup_{i=1}^{n} S_i$. 回答若干次询问, 每次询问给定元素 $x \in S$, 回答包含该元素的集合标号 k (即存在且唯一的一个 k 使得 $S_k \ni x$).

暂不谈实现时的内存效率, 我们可以通过二分或者二进制分组将 问题 2.1 转化为一个更简单的问题:

> **问题 2.2**
>
> 给定 $n = 2$ 个无交集 S^+, S^-, 设 $S = S^+ \sqcup S^-$. 回答若干次询问, 每次询问给定元素 $x \in S$, 判断是否 $x \in S^+$.

问题 2.2 明显不强于 问题 1.1, 因此可以使用 Bloom Filter 解决: 对 S^+ (或 S^-) 建立 Bloom Filter, 查询是否有 $x \in S^+$ (或 $x \in S^-$) 即可得到 问题 2.2 解决方法. 然而, Bloom Filter 无法利用 $x \in S = S^+ \sqcup S^-$ 这一信息, 而 Four-colored Filter 则是充分利用这一点来对算法进行了改进.

> **算法 2.3 (Four-colored Filter)**
>
> 对 问题 2.2, 取定 hash 值域 n 和 hash 函数 $h : S \to [1:n]^2$, 以此生成无向图 $G = (V, E)$, 其中 $V := [1:n]$,
>
> $$E^+ := \{(u, v, +) : (u, v) \in h(S^+)\}, \quad E^- := \{(s, t, -) : (s, t) \in h(S^-)\};$$
> $$E := E^+ \sqcup E^-.$$
>
> 称 E^+ 的元素为实边, E^- 的元素为虚边. 不妨 $|S^+| \geq |S^-|$, 反复尝试构造四染色 $c : V \to \{0, 1, 2, 3\}$, 满足
>
> - 实边异色: $\forall (u, v, +) \in E^+,\ c(u) \neq c(v)$;
> - 虚边同色: $\forall (s, t, -) \in E^-,\ c(u) = c(v)$.
>
> 若成功得到 c, 则 Four-colored Filter 建立完成. 此后
>
> - 查询元素 y 时: 计算 $h(y) = (a, b)$, 若 $c(a) = c(b)$, 则 $y \in S^-$, 否则 $y \in S^+$.
> - (若要求 Four-colored Filter 在线) 插入元素 x 时: 在原图基础上加入 x 对应的实边或虚边, 并从其两端出发搜索调整染色方案直到 c 再次合法.

相比于 Bloom Filter, 一旦 Four-colored Filter 成功建立, 它的回答都一定是快速且完全正确的, 它适合用于应对对固定数据的大规模查询. 不过, 由于 4-coloring 问题是 NPC 的, 对 c 的寻找必然依赖于随机性算法. 此外, h 给出的 G 本身可能带有明显的矛盾: 若虚边连通块中出现了实边, 则 c 不可能存在, 这要求我们对 h 的随机选取或调整. 事实上, 制定染色策略时认定 $|S^+| \geq |S^-|$ 也正是因此: 大量的同色连边倾向于在 G 中形成一个巨大的连通块, 且此连通块在缩点后的度数较大, 二者分别容易导致块内和块外的染色不存在; 将较小的 E^- 作为同色边则倾向于让 G 的连通块和缩点后度数更加均匀, 更容易成功染色.

笔者认为值得探索的地方:

1. n 的选取. 从信息熵粗略估计, 当 $n = \sqrt{|S|/2}$ 时就已经能够无损地存放 S^+ 和 S^- 的信息. 一般实现的时候取的是 $n = |S|$ (此处存疑), 这是否有必要?
2. 染色数 k 的选取. 我们直接承认了 $k = 4$, 这是因为 k 更小时染色难以完成, k 更大时会带来更大空间开销. 如何更细致地描述并推导 k 的选取?

3 Fermat Sketch

所谓 "sketch", 即 "草图", 指的是对数据的一种压缩存储方式. Fermat Sketch 试图解决以下问题:

> **问题 3.1**
>
> 假设云端向本地用户传输了若干份数据包, 每个数据包有相应的 ID 和包数. 传输完成后, 要求云端向本地附带一则 sketch 信息, 帮助用户检查传输完整性 (是否丢包, 有哪几份数据包发生丢包).

我们可以将 (ID, 包数) 集合视为一个关于 ID 的可重集, 目标便是对两个可重集进行快速比较, 尽可能地描述出它们的差异.

> **算法 3.2 (Fermat Sketch)**
>
> 对 问题 3.1, 设 ID 的值域 $I = [1:m]$, 总包数 q. 选定 hash 范围 n, 两个 hash 函数 $f, g : I \to [1:n]$ 和素数 $p > \max\{m, q\}$. 定义一个 Abel 群 $G = (\mathbb{F}_p \times \mathbb{Z}, +)$, 其中
>
> $$+ : (u, a) \mapsto (v, b) \mapsto (u +_{\mathbb{F}_p} v, a +_{\mathbb{N}} b).$$
>
> 本地和云端分别维护两份 sketch $\{s_i\}_{i=1}^n, \{t_i\}_{i=1}^n$, 其中 $s_i, t_i \in G$. 当接收到 ID 为 i 的包时, 令
>
> $$s_{f(i)} \leftarrow s_{f(i)} + (i, 1), \quad t_{g(i)} \leftarrow t_{g(i)} + (i, 1).$$
>
> 传输完成后, 云端将自己的两份 sketch 提供给本地. 不妨设此时云端 sketch 为 $\{s_i\}, \{t_i\}$, 本地 sketch 为 $\{s'_i\}, \{t'_i\}$. 接下来, 不断寻找下标 i 使得 $s_i \neq s'_i$ 或者 $t_i \neq t'_i$, 不妨设找到某个 $s_i \neq s'_i$, 计算
>
> $$(w, c) := s_i - s'_i, \quad i_? := w \times c^{-1} \in \mathbb{F}_p.$$
>
> 检查是否有 $f(i_?) = i$ (rehash verification), 讨论:
>
> - 若是, 相信 (w, c) 恰好描述了 ID 为 $i_?$ 的包的本地-云端差异. 此时 c 就是差异数量. 此后在本地 sketch 中消除此差异, 令
>
> $$s'_i \leftarrow s_i, \quad t'_{g(i_?)} \leftarrow t'_{g(i_?)} + (i, c).$$
>
> - 否则, 忽略这个 i, 寻找其他的 i.
>
> 如果一切 $s_i = s'_i$ 且 $t_i = t'_i$, Fermat Sketch 便认为成功找出了所有差异; 否则若找不到任何一个合适的 i, 则认为 Fermat Sketch 的搜索失败.

可以感知到, 用两个 hash 维护两份 sketch 的目的是让 "消除差异" 变得可能: 例如某处 s_i 的 hash 冲突导致无法通过 s_i 还原差异, 我们还有 t_i 能提供线索; 如果某处 s_i 可以用作还原差异, 它也能为对应的 $t_{g(i_?)}$ 提供修正, 如此迭代便提高了顺利找出所有差异的概率.

如果希望提高 rehash verification 的置信度, 只需要对每个 ID 附带上随机指纹 $fp \in \mathbb{F}_p$, 参与与 ID 相同的运算, rehash 时顺便比较随机指纹即可.

4 补充与思考

本文尚有大量未提及的经典 PDS, 例如 Count-min Sketch, Universal Sketch 等. PDS 在诸多计算机领域有所应用, 如垂直领域大模型的检索增强生成, 缓存替换策略等, 不一而足.

笔者认为, PDS 有很大的研究价值. 理论上, 一切问题都可以通过附带一个 "错误接受程度" 来引入 PDS; 工业上, 大多数问题都并不要求精确求解, 因此 PDS 具有很强的普适性和实用性. 受 Four-colored Filter 的启发, 笔者猜测一种可能的研究方向是将某些问题先映射到一个 NPC 但足以提供良好性质的问题上, 进而借助已有的对 NPC 问题的丰富研究来设计更多 PDS.

References

1. 《信息科学技术概论》2024/09/13 课程内容 (by 杨仝老师).
2. https://en.wikipedia.org/wiki/Bloom_filter